Unsupervised Deep Learning for Multi-Genre Music Generation: A Progressive Pipeline from Autoencoders to RLHF-Tuned Transformers

Farhan Taukir Rafin
ID: 22299207
Department of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh

Jamshedul Alam Khan Hridoy
ID: 22201680
Department of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh

Abstract

This paper presents a comprehensive pipeline for unsupervised music generation using deep neural networks, developed as part of the CSE425: Neural Networks course. The system progresses through four increasingly sophisticated tasks: (1) an LSTM-based autoencoder for short sequence reconstruction, (2) a Variational Autoencoder (VAE) for multi-genre diversity with reparameterization, (3) a Transformer decoder for long-form autoregressive generation, and (4) a Reinforcement Learning from Human Feedback (RLHF) fine-tuning stage that aligns generated music with human preference scores. Without any explicit genre labels, the models learn meaningful musical representations from MIDI data drawn from the MAESTRO v2 and Lakh MIDI datasets. Quantitative evaluation across pitch diversity, rhythm diversity, repetition ratio, and perplexity demonstrates progressive improvement across all tasks. The final RLHF-tuned Transformer achieves a mean reward of 4.1 out of 5 and a 3.4$\times$ improvement over random baselines, producing coherent and musically structured compositions. All models are implemented in PyTorch and designed for efficient execution on consumer-grade GPUs.

1 Introduction

Music generation is a challenging sequence modeling problem that requires a model to learn long-range temporal dependencies, stylistic coherence, and structural regularity simultaneously. Unlike image or text generation, music exhibits hierarchical structure across multiple timescales: individual note onsets, bar-level melodic phrases, and section-level form. Developing models that capture these properties without explicit supervision—that is, without labeled genre or style tags—remains an open research problem.

This project addresses unsupervised music generation through a four-stage curriculum. Each stage introduces a new architectural element or training objective that expands the generative capability of the system. The primary contributions are as follows:

- An LSTM autoencoder that learns a compact latent representation of piano-roll sequences from a single genre, providing a baseline for reconstruction quality.
- A Variational Autoencoder (VAE) extended to multi-genre data, enabling smooth interpolation in latent space and increased output diversity.
- A Transformer decoder trained autoregressively over event-tokenized MIDI, capable of generating sequences of up to 512 tokens with measurable improvement in perplexity.
- A policy-gradient fine-tuning stage (RLHF) that incorporates human preference scores into the training signal, yielding measurable gains in pitch diversity, rhythm diversity, and repetition reduction.

The paper is structured as follows. Section 2 reviews related work. Section 3 describes the datasets and preprocessing pipeline. Sections 4–7 detail each model in turn. Section 8 defines the evaluation metrics. Section 9 presents quantitative and qualitative results alongside baseline comparisons. Section 10 discusses engineering challenges and solutions. Section 13 concludes with directions for future work.

2 Related Work

Recurrent Models for Music. Early neural approaches to music generation employed recurrent networks. Eck and Schmidhuber **?** demonstrated that LSTMs could learn short-term musical structure from MIDI. Subsequent work by Boulanger-Lewandowski et al. **?** applied recurrent temporal restricted Boltzmann machines to polyphonic modeling, while MusicRNN **?** introduced hierarchical representations with two-level LSTMs to handle longer sequences. These approaches form the conceptual basis for the autoencoder architecture used in Task 1.

Variational Autoencoders for Music. MusicVAE **?** is the seminal application of VAEs to music, using a hierarchical decoder to condition bar-level generation on a global latent code. The β-VAE framework **?** introduced a weighting coefficient β on the KL divergence term to encourage disentangled latent representations, a technique we apply via KL annealing in Task 2. Other work, such as Music FaderNets **?**, has explored attribute-conditioned generation within a VAE framework.

Transformer-Based Generation. Music Transformer **?** extended the Transformer architecture **?** to symbolic music by introducing relative positional encodings that better capture the repetitive structure of musical phrases. MuseNet **?** and Jukebox **?** scale these ideas to large multi-genre models. Our Task 3 uses a standard Transformer decoder with absolute positional encoding, prioritizing interpretability and resource efficiency over raw scale.

Reinforcement Learning from Human Feedback. RLHF was introduced in the context of language models by Ziegler et al. **?** and later popularized by InstructGPT **?**. Its application to creative generation domains is more recent; Cideron et al. **?** demonstrated reward-guided music generation using human raters. Our Task 4 adopts the REINFORCE algorithm **?** with a lightweight reward model calibrated from a small human survey, an approach well-suited to the resource constraints of a course project.

3 Datasets and Preprocessing

3.1 MAESTRO v2

The MIDI and Audio Edited for Synchronous TRacks and Organization (MAESTRO) dataset, version 2 **?**, contains approximately 200 hours of virtuosic piano performances recorded at international piano competitions. Files are aligned MIDI and audio recordings at millisecond precision. For this project, only the MIDI files are used. MAESTRO serves as the primary training corpus for Task 1 and provides the classical piano component for Tasks 2–4.

3.2 Lakh MIDI Dataset

The Lakh MIDI Dataset **?** contains approximately 176,581 unique MIDI files sourced from the internet, spanning jazz, pop, rock, electronic, and classical genres. The multi-genre coverage is critical for Task 2 onward, where the model must generalize across stylistic variation without explicit genre supervision.

3.3 Piano-Roll Representation

For Tasks 1 and 2, MIDI files are converted to a binary piano-roll matrix $\mathbf{X} \in \{0,1\}^{88\times T}$, where 88 corresponds to the standard MIDI pitch range and T is a fixed window of 512 time steps (each step corresponding to a 32nd note at a reference tempo). This representation is frame-based and well-suited to reconstruction objectives.

3.4 Event-Based Tokenization

For Tasks 3 and 4, a compact event vocabulary is used. Each MIDI event is mapped to one of 91 tokens: 88 note-on events (one per pitch), a note-off token, a time-shift token, and a velocity token. This event-based tokenization is lossless with respect to onset and offset timing and reduces the average sequence length compared to piano-roll representations, enabling the Transformer to model longer musical contexts within a fixed window.

3.5 Data Splits

All datasets are split 80/10/10 into training, validation, and test sets using a fixed random seed to ensure reproducibility. A `SplitManager` class records the file-level partition, preventing cross-contamination between splits.

4 Task 1: LSTM Autoencoder

4.1 Architecture

The LSTM Autoencoder consists of an encoder that compresses a variable-length piano-roll sequence to a fixed-dimensional latent vector, and a decoder that reconstructs the original sequence from that vector. Formally, let the input be $\mathbf{X} \in \mathbb{R}^{B\times T\times 88}$, where B is the batch size and T is the sequence length.

Encoder. A two-layer LSTM with hidden sizes 512 and 256 processes $\mathbf{X}$ step-by-step. The final hidden state of the second layer is projected by a linear layer to produce the latent code $\mathbf{z} \in \mathbb{R}^{128}$.

Decoder. A learned linear layer expands $\mathbf{z}$ to the initial hidden state of a two-layer LSTM with hidden sizes 256 and 512. The decoder LSTM is unrolled for T steps and the output at each step is passed through a sigmoid activation to produce a binary piano-roll prediction $\hat{\mathbf{X}} \in [0,1]^{B\times T\times 88}$.

Training Objective. The model is trained with mean squared error (MSE) reconstruction loss:

$$\mathcal{L}_{\text{recon}} = \frac{1}{BT\cdot 88}\sum_{b,t,p}\left(X_{b,t,p} - \hat{X}_{b,t,p}\right)^2. \tag{1}$$

4.2 Training Details

The model is trained for 50 epochs with batch size 32 using the Adam optimizer at a learning rate of 10^{-3}. A learning rate scheduler reduces the rate by a factor of 0.5 if validation loss does not improve for 5 consecutive epochs. The best checkpoint is selected based on validation loss.

4.3 Results

The reconstruction loss converges smoothly, decreasing from approximately 0.42 to a final value of 0.15. The 128-dimensional bottleneck successfully captures the harmonic and rhythmic content of

short piano sequences. Five generated MIDI files were produced by sampling from the encoder's latent space and decoding with the learned decoder, each exhibiting recognizable melodic patterns consistent with the MAESTRO training distribution.

5 Task 2: Variational Autoencoder

5.1 Architecture

The VAE extends the Task 1 autoencoder by replacing the deterministic latent code with a stochastic latent variable $\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}, \mathrm{diag}(\boldsymbol{\sigma}^2))$. The encoder now outputs both a mean vector $\boldsymbol{\mu} \in \mathbb{R}^{128}$ and a log-variance vector $\log \boldsymbol{\sigma}^2 \in \mathbb{R}^{128}$. Sampling is performed via the reparameterization trick:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \tag{2}$$

This allows gradients to flow through the sampling operation during backpropagation.

5.2 Training Objective

The VAE is trained to maximize the Evidence Lower BOund (ELBO):

$$\mathcal{L}_{\mathrm{ELBO}} = \mathbb{E}_{q(\mathbf{z}|\mathbf{X})} \left[\log p(\mathbf{X}|\mathbf{z})\right] - \beta \cdot D_{\mathrm{KL}}(q(\mathbf{z}|\mathbf{X}) \,\|\, p(\mathbf{z})) \,, \tag{3}$$

where $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ is the standard normal prior and β is the KL weighting coefficient. The KL divergence has a closed form:

$$D_{\mathrm{KL}} = -\frac{1}{2} \sum_{j=1}^{128} \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right). \tag{4}$$

5.3 KL Annealing

To prevent posterior collapse—where the model ignores the latent code and sets $\boldsymbol{\sigma}^2 \to \mathbf{1}$ uniformly—β is linearly annealed from 0 to 1 over the first 20 epochs of training, following the schedule of Bowman et al. **?**. This allows the reconstruction term to dominate early in training, giving the decoder time to learn useful mappings before the KL constraint becomes binding.

5.4 Multi-Genre Training

MAESTRO and a curated subset of the Lakh MIDI Dataset are combined for training. No genre labels are provided to the model; it must organize its latent space based solely on musical content. The resulting latent space exhibits genre-related clustering when visualized with t-SNE, indicating that the model implicitly discovers stylistic structure.

5.5 Latent Interpolation

To verify the smoothness of the learned representation, linear interpolation between two latent codes $\mathbf{z}_1$ and $\mathbf{z}_2$ from different compositions is decoded at 8 equally spaced steps:

$$\mathbf{z}_\alpha = (1 - \alpha)\mathbf{z}_1 + \alpha \mathbf{z}_2, \quad \alpha \in \{0, 1/7, 2/7, \ldots, 1\}. \tag{5}$$

The resulting decoded sequences transition smoothly in character, confirming that the VAE has organized musical concepts continuously in latent space rather than in a fragmented, discontinuous manner.

5.6 Results

Eight MIDI samples were generated by sampling from the prior $p(\mathbf{z})$. The ELBO loss stabilizes after approximately 35 epochs. The final reconstruction component is 0.22, lower than the Task 1 value of 0.15 at convergence—a difference attributable to the increased data diversity from multi-genre training rather than model degradation. Human evaluation of interpolated sequences confirms smooth stylistic transitions across genres.

6 Task 3: Transformer-Based Generator

6.1 Architecture

The Transformer decoder **?** models the joint distribution of a token sequence $p(x_1, \ldots, x_T) = \prod_{t=1}^{T} p_\theta(x_t | x_{<t})$ autoregressively. The architecture comprises the following components:

- **Token Embedding:** $\mathbf{E} \in \mathbb{R}^{91 \times 256}$ maps each of the 91 vocabulary tokens to a 256-dimensional embedding vector.
- **Positional Encoding:** Sinusoidal positional encodings are added to the embeddings, following the original Transformer formulation.
- **Decoder Layers:** 6 stacked layers, each containing masked multi-head self-attention (8 heads, $d_k = 32$), a position-wise feed-forward network (FFN dimension 1024), and layer normalization with residual connections.
- **Output Projection:** A linear layer maps the final hidden state to $\mathbb{R}^{91}$, followed by a softmax for next-token probability estimation.

The total parameter count is approximately 18 million, predominantly in the FFN sub-layers.

6.2 Tokenization

The event-based vocabulary (Section 3) encodes MIDI as a sequence of tokens. A sequence of length 512 covers approximately 4 bars of music at a moderate tempo. During training, sequences are windowed with a stride of 256 tokens to maximize data utilization.

6.3 Training Details

- **Loss:** Cross-entropy with label smoothing ($\alpha = 0.05$) to regularize the output distribution and prevent overconfident predictions.
- **Optimizer:** AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay 10^{-2}.
- **Learning Rate Schedule:** Linear warmup for 1000 steps followed by cosine decay to 10^{-5}.
- **Batch Size:** 32 sequences.
- **Hardware:** NVIDIA T4 GPU (14.6 GB VRAM) on Kaggle Notebooks.

Checkpoints are saved every epoch, and the model with the lowest validation loss is used for evaluation.

6.4 Inference

At generation time, the model is prompted with a start token and samples autoregressively using temperature-scaled softmax ($\tau = 0.9$) combined with top-k filtering ($k = 40$). This hyperparameter combination balances output diversity against musical coherence, a trade-off studied empirically on the validation set.

6.5 Results

Validation perplexity decreases from approximately 45.2 at epoch 1 to 12.1 at convergence, indicating that the model learns to assign significantly higher probability mass to musically plausible continuations. Ten generated MIDI files, each containing 512 tokens ($\approx$ 2–3 minutes of music), demonstrate clear learned pitch progressions, recurring rhythmic motifs, and phrase-level structure. Inference latency is approximately 120 milliseconds per sequence on the T4 GPU.

7 Task 4: RLHF Human Preference Tuning

7.1 Motivation

Standard cross-entropy training optimizes for the statistical likelihood of training data but does not directly optimize for perceptual quality. Human listeners evaluate music on criteria that are difficult

to capture with token-level losses, such as rhythmic consistency, harmonic interest, and the absence of excessive repetition. Reinforcement Learning from Human Feedback (RLHF) addresses this gap by incorporating human evaluation signals into the training objective.

7.2 Reward Model

A lightweight reward function $R : \tau \to [1, 5]$ is defined over generated token sequences τ. It aggregates three automated proxies calibrated against human survey scores:

1. **Pitch Diversity** δ_p: the fraction of the 88-dimensional pitch range that appears in the sequence.
2. **Rhythm Diversity** δ_r: the ratio of unique inter-onset intervals to total transitions.
3. **Repetition Ratio** ρ: the fraction of 4-gram patterns that appear more than once.

The combined quality score is:

$$q = w_p \cdot \delta_p + w_r \cdot \delta_r - w_\rho \cdot \rho, \tag{6}$$

where $w_p = w_r = 0.4$ and $w_\rho = 0.2$. The raw score q is linearly mapped to the $[1, 5]$ range:

$$R = R_{\min} + (R_{\max} - R_{\min}) \times \frac{q - q_{\min}}{q_{\max} - q_{\min}}. \tag{7}$$

The calibration constants $q_{\min}$, $q_{\max}$ are estimated from 100 held-out generations, and the weights are set to maximize Spearman rank correlation with the 10-participant human survey scores.

7.3 Policy Gradient Update

Fine-tuning proceeds with the REINFORCE algorithm **?**. At each training step, $N = 16$ sequences are sampled from the current policy π_θ, rewards $\{R_i\}_{i=1}^N$ are computed, and the policy is updated by:

$$\nabla_\theta \mathcal{L}_\pi = -\frac{1}{N} \sum_{i=1}^{N} A_i \cdot \nabla_\theta \log \pi_\theta(\tau_i), \tag{8}$$

where the advantage A_i is computed with a running baseline to reduce variance:

$$A_i = \frac{R_i - \bar{R}}{\mathrm{std}(\{R_j\}) + \epsilon}, \quad \epsilon = 10^{-8}. \tag{9}$$

7.4 Training Configuration

Table 1: RLHF fine-tuning hyperparameters.

Parameter	Value
Base model	Task 3 best checkpoint
RLHF epochs	10
Rollouts per update	16
Learning rate	10^{-5}
Gradient clip norm	1.0
Optimizer	AdamW
Hardware	NVIDIA T4 (14.6 GB)

A conservative learning rate of 10^{-5} prevents catastrophic forgetting of the pretrained representations. Per-sequence gradient accumulation is used during rollout collection to avoid retaining full computation graphs in GPU memory.

7.5 Results

Over 10 RLHF epochs, the mean reward increases from 3.5 to 4.1, a relative improvement of approximately 17%. The component-wise changes are:

- Pitch diversity: $+5$–10%
- Rhythm diversity: $+8$–12%
- Repetition ratio: -10–15%

Human survey participants (10 evaluators, 10 samples per task, 5-point Likert scale) confirm a statistically consistent preference for RLHF-tuned outputs over Task 3 outputs across the dimensions of quality, coherence, diversity, and musicality.

8 Evaluation Metrics

We evaluate all models using a shared set of metrics, enabling direct comparison across tasks and against non-learning baselines.

Pitch Histogram Distance. Measures the L_1 deviation of the generated pitch class distribution from a reference corpus distribution:

$$H(p, q) = \sum_{i=0}^{11} |p_i - q_i|, \tag{10}$$

where p_i and q_i are the normalized frequencies of pitch class i in the generated and reference sets, respectively. Lower values indicate more natural pitch usage.

Rhythm Diversity. Quantifies the variety of temporal patterns in a sequence:

$$D = \frac{|\{\text{unique inter-onset intervals}\}|}{|\{\text{all transitions}\}|}. \tag{11}$$

Higher values indicate more varied rhythmic content.

Repetition Ratio. Captures the tendency to repeat short patterns:

$$R = \frac{\text{count of repeated } n\text{-grams}}{\text{total } n\text{-grams}}, \quad n = 4. \tag{12}$$

Lower values indicate less mechanical repetition.

Perplexity. For the Transformer model (Tasks 3 and 4), validation perplexity measures how well the model predicts held-out token sequences:

$$\text{PPL} = \exp\left(\frac{1}{T}\sum_{t=1}^{T} -\log p_\theta(x_t \mid x_{<t})\right). \tag{13}$$

Lower perplexity indicates better sequence modeling.

Human Preference Score. A 5-point Likert scale survey administered to 10 participants, rating generated samples on quality, coherence, diversity, and musicality. The mean score across participants and dimensions constitutes the human preference metric.

9 Experimental Results and Baseline Comparison

9.1 Non-Learning Baselines

Two baselines are implemented to provide lower bounds on performance:

Random Generator: Samples each pitch independently with probability proportional to the observed pitch frequency in the training set. This baseline has no temporal coherence and serves as the floor.

Markov Chain Generator: A first-order transition matrix is estimated from MAESTRO, capturing local pitch-to-pitch dependencies. While it reproduces some local melodic contours, it lacks any long-range structure.

9.2 Quantitative Comparison

Table 2 summarizes the performance of all methods across evaluation metrics.

Table 2: Performance comparison across all methods. Higher pitch diversity and rhythm diversity are better; lower repetition ratio is better. Human reward is on a 5-point scale.

Method	Pitch Diversity	Rhythm Diversity	Rep. Ratio	Mean Reward
Random Generator	0.15	0.08	0.71	1.2
Markov Chain	0.28	0.22	0.54	1.8
Task 1 (LSTM AE)	0.42	0.35	0.39	2.3
Task 2 (VAE)	0.51	0.44	0.32	2.8
Task 3 (Transformer)	0.68	0.59	0.24	3.5
Task 4 (RLHF)	**0.73**	**0.68**	**0.21**	**4.1**

9.3 Model Complexity and Resource Usage

Table 3 reports the computational profile of each model to contextualize the performance gains in terms of resource cost.

Table 3: Model complexity and resource usage.

Task	Model	Parameters	VRAM Usage	Inference Latency
1	LSTM Autoencoder	1.2M	4 GB	50 ms
2	VAE	1.3M	5 GB	55 ms
3	Transformer	18M	9 GB	120 ms
4	RLHF-Tuned Transf.	18M	14 GB	125 ms

9.4 Qualitative Observations

The progression of model quality is consistent with qualitative listening tests:

Task 1: Generated sequences are tonally stable and closely mirror the harmonic language of MAESTRO. Diversity is limited because the latent space is constrained to a single genre and lacks stochasticity.

Task 2: The VAE introduces sample variety that is absent from the deterministic autoencoder. Interpolated sequences exhibit gradual stylistic transitions, suggesting a meaningful organization of the latent space despite the absence of genre labels.

Task 3: The Transformer generates the most structurally coherent music. Recurring motifs appear at irregular but musically plausible intervals. Pitch progressions follow functional harmonic patterns consistent with both classical and popular styles present in the training data.

Task 4: The RLHF-tuned model maintains the coherence of Task 3 while exhibiting fewer verbatim repetitions and more varied rhythmic patterning. Survey participants specifically note improved rhythmic interest and reduced monotony compared to Task 3 outputs.

10 Challenges and Solutions

10.1 GPU Memory Constraints

The NVIDIA T4 GPU used in this project has 14.6 GB of VRAM, which is sufficient for individual forward and backward passes but becomes a bottleneck during RLHF rollout collection. Two mitigation strategies are employed:

1. **CPU Deserialization:** Model checkpoints are loaded onto CPU memory first using `map_location='cpu'` in PyTorch, then transferred to the GPU. This avoids allocating a second copy of the model on the GPU during checkpoint loading.
2. **Per-Sequence Gradient Accumulation:** During RLHF rollout collection, gradients are computed and accumulated one sequence at a time rather than over the full batch, preventing the computation graph from accumulating in memory.

A reduced-resource fallback configuration with smaller batch sizes is included in `src/config.py` for users with less available VRAM.

10.2 Multi-Genre Data Integration

The MAESTRO dataset covers only classical piano repertoire, while Lakh MIDI spans multiple genres with heterogeneous MIDI instrumentation and tempo conventions. Combining these datasets without label supervision risks the model collapsing to a single dominant style. The VAE's KL annealing schedule was found to be critical for maintaining diversity: without annealing, the model tends to produce a single average style rather than genre-specific clusters.

10.3 Sequence Length and Memory Trade-off

Longer sequences improve musical coherence but increase the memory footprint of the Transformer's attention matrices quadratically. The window size of 512 tokens was selected as the maximum length that fits within the T4's VRAM budget at batch size 32. During evaluation, longer compositions are generated autoregressively by re-conditioning on the most recent 512 tokens at each step.

10.4 RLHF Training Stability

Policy gradient methods are known to suffer from high-variance gradient estimates and potential reward hacking. The following measures were taken to stabilize training:

- **Advantage Normalization:** Per-batch advantage standardization reduces the scale of gradient updates.
- **Gradient Clipping:** The ℓ_2 norm of gradients is clipped to a maximum of 1.0, preventing catastrophic updates.
- **Conservative Learning Rate:** 10^{-5} is substantially smaller than the pretraining rate, limiting the magnitude of policy changes per update.
- **Reward Calibration:** The reward function is bounded to $[1, 5]$ by construction, preventing extreme reward signals.

10.5 Reproducibility

All stochastic operations are seeded at the start of each script using a single global seed applied to Python's `random` module, NumPy, and PyTorch (including the CUDA backend where applicable). Best-checkpoint selection is performed deterministically based on validation loss, and MIDI export uses a fixed quantization grid to ensure byte-identical output files given the same latent code.

11 Implementation Architecture

The project is organized as a Python package with strict separation of concerns. All hyperparameters are centralized in `src/config.py`, making it straightforward to reproduce any experiment

by loading the corresponding configuration dictionary. Model definitions (`src/models/`), training loops (`src/training/`), preprocessing utilities (`src/preprocessing/`), generation scripts (`src/generation/`), and evaluation metrics (`src/evaluation/`) are each encapsulated in separate modules with consistent public interfaces.

Training loops are instrumented with `tqdm` progress bars and emit loss/metric summaries at configurable intervals. Validation is performed with `torch.no_grad()` context to suppress gradient computation and reduce memory overhead. All generated MIDI files are exported to the `outputs/generated_midis/` directory using `pretty_midi`, and all training curves are saved to `outputs/plots/` using `matplotlib`.

Each task is also available as a self-contained Jupyter notebook under `notebooks/`, enabling execution on Kaggle with a single kernel run. The notebooks share the same configuration and model definitions as the Python package, ensuring consistency between interactive and batch execution modes.

12 Future Work

Several extensions of the current work are of interest:

Architecture Enhancements. Relative positional encodings **?** would allow the Transformer to better capture the relative distance between events, which is more musically meaningful than absolute position. A hierarchical VAE **?** with separate latent codes for bar-level and phrase-level structure could produce more globally coherent long compositions. Attention weight visualization would provide interpretability into which prior events influence each generated token.

Training Improvements. Curriculum learning, where training begins on short, harmonically simple sequences and progressively increases difficulty, may improve convergence speed and final quality. Adversarial training with a discriminator conditioned on genre or style would provide a richer training signal than reconstruction loss alone. A multi-task auxiliary loss for genre classification could encourage more structured latent organization.

Evaluation Extensions. Music theory metrics such as voice leading quality, harmonic progression coherence, and melodic contour smoothness would provide more nuanced evaluation. A larger-scale perceptual study with trained musicians would offer a more reliable human preference signal than the 10-participant survey used here. Comparison against published state-of-the-art systems such as MusicVAE **?** and Music Transformer **?** would contextualize the absolute level of quality achieved.

Application Development. The generative models developed here could be deployed in a browser-based interface for real-time interactive music generation. Style transfer between genres, enabled by the VAE's latent interpolation capability, could be packaged as a creative tool for musicians. Music completion and inpainting tasks—where a fragment of an existing composition is provided as context—are a natural extension of the autoregressive generation framework.

13 Conclusion

This paper has presented a progressive pipeline for unsupervised multi-genre music generation. Starting from a simple LSTM autoencoder, each task introduces a principled improvement: stochasticity and multi-genre training in the VAE, long-range modeling in the Transformer, and human preference alignment via RLHF. The final system achieves a mean human preference score of 4.1/5 and a 3.4$\times$ improvement over a random pitch-sampling baseline, demonstrating that the full pipeline produces genuinely musical output.

From an educational standpoint, the four-task structure effectively illustrates the trajectory of modern generative modeling: from deterministic bottleneck representations, through latent variable models with tractable inference, to sequence-to-sequence architectures and reinforcement fine-tuning. Each component builds directly on the previous, making the progression conceptually transparent and the ablation analysis straightforward.

The memory-efficient implementation, designed for the resource constraints of the Kaggle T4 environment, demonstrates that research-quality music generation is achievable without access to large-scale compute infrastructure. All code, trained checkpoints, and generated MIDI files are available in the project repository.

Acknowledgments and Disclosure of Funding

The authors thank the instructors and teaching assistants of CSE425: Neural Networks at BRAC University for their guidance throughout this project. The MAESTRO dataset was provided by Google Brain. The Lakh MIDI Dataset was compiled by Colin Raffel as part of his doctoral work at Columbia University. Experiments were conducted using free compute resources provided by Kaggle Notebooks.

References

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.

[2] Roberts, A., Engel, J., Raffel, C., Hawthorne, C., & Eck, D. (2018). A hierarchical latent vector model for learning long-term structure in music. *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pp. 4364–4373.

[3] Huang, C.Z.A., Vaswani, A., Uszkoreit, J., Shazeer, N., Simon, I., Hawthorne, C., Dai, A.M., Hoffman, M.D., Dinculescu, M., & Eck, D. (2018). Music Transformer: Generating music with long-term structure. *International Conference on Learning Representations (ICLR)*.

[4] Hawthorne, C., Stasyuk, A., Roberts, A., Simon, I., Huang, C.Z.A., Dieleman, S., Eck, D. (2018). Enabling factorized piano music modeling and generation with the MAESTRO dataset. *International Conference on Learning Representations (ICLR)*.

[5] Raffel, C. (2016). Learning-based methods for comparing sequences, with applications to audio-to-MIDI alignment and matching. *PhD thesis, Columbia University*.

[6] Higgins, I., Matthey, L., Pal, A., Burgess, C., Glorot, X., Botvinick, M., Mohamed, S., & Lerchner, A. (2017). β-VAE: Learning basic visual concepts with a constrained variational framework. *International Conference on Learning Representations (ICLR)*.

[7] Kingma, D.P. & Welling, M. (2013). Auto-encoding variational Bayes. *arXiv preprint arXiv:1312.6114*.

[8] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., ... & Lowe, R. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35.

[9] Williams, R.J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4), 229–256.

[10] Bowman, S.R., Vilnis, L., Vinyals, O., Dai, A., Jozefowicz, R., & Bengio, S. (2016). Generating sentences from a continuous space. *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning*, pp. 10–21.

[11] Dhariwal, P., Jun, H., Payne, C., Kim, J.W., Radford, A., & Sutskever, I. (2020). Jukebox: A generative model for music. *arXiv preprint arXiv:2005.00341*.

[12] Eck, D. & Schmidhuber, J. (2002). Finding temporal structure in music: Blues improvisation with LSTM recurrent networks. *Proceedings of the 2002 IEEE Workshop on Neural Networks for Signal Processing*, pp. 747–756.

[13] Boulanger-Lewandowski, N., Bengio, Y., & Vincent, P. (2012). Modeling temporal dependencies in high-dimensional sequences: Application to polyphonic music generation and transcription. *Proceedings of the 29th International Conference on Machine Learning (ICML)*, pp. 1159–1166.

[14] Ziegler, D.M., Stiennon, N., Wu, J., Brown, T.B., Radford, A., Amodei, D., ... & Irving, G. (2019). Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*.

[15] Cideron, G., Girgin, S., Verzetti, M., Kastner, D., Ramos, G., Nauleau, M., ... & Pietquin, O. (2024). MusicRL: Aligning music generation to human preferences. *arXiv preprint arXiv:2402.04229*.

A Project File Structure

The complete project repository is organized as follows:

```
music-generation-unsupervised/
|-- src/
|   |-- config.py                  # All hyperparameters
|   |-- models/
|   |   |-- autoencoder.py         # LSTM AE (Task 1)
|   |   |-- vae.py                 # VAE (Task 2)
|   |   |-- transformer.py         # Transformer (Task 3)
|   |   +-- diffusion.py           # RLHF wrapper (Task 4)
|   |-- training/
|   |   |-- train_ae.py            # Task 1 training loop
|   |   |-- train_vae.py           # Task 2 training loop
|   |   |-- train_transformer.py   # Task 3 training loop
|   |   +-- rlhf_finetune.py       # Task 4 fine-tuning
|   |-- preprocessing/
|   |   |-- midi_parser.py
|   |   |-- piano_roll.py
|   |   |-- tokenizer.py           # Event tokenization
|   |   +-- split_manager.py
|   |-- generation/
|   |   |-- generate_music.py
|   |   +-- midi_export.py
|   +-- evaluation/
|       +-- metrics.py
|-- notebooks/
|   |-- task1_autoencoder.ipynb
|   |-- task2_vae.ipynb
|   |-- task3_transformer.ipynb
|   |-- task4_rlhf.ipynb
|   +-- baseline_markov.ipynb
|-- outputs/
|   |-- checkpoints/
|   |-- generated_midis/
|   |-- plots/
|   +-- survey_results/
|-- data/
|   |-- raw_midi/
|   |-- processed/
|   +-- train_test_split/
|-- report/
|-- README.md
+-- requirements.txt
```